JAGAT GURU NANAK DEV PUNJAB STATE OPEN UNIVERSITY, PATIALA

DEPARTMENT OF COMPUTER SCIENCE & APPLICATIONS

M.Sc. (Computer Science) — Semester-IV

SEMINAR VIVA VOCE & QUESTION DEFENSE GUIDE

Course Code: MSCS-4-04 | Seminar (M.Sc. Computer Science, Semester-IV)

Topic: Architectural Patterns for Offline-First Distributed Web Applications: Balancing Zero-Latency User Experience with Eventual Consistency
Evaluator Target: University Examiners, Faculty Chair, Seminar Panel
Q&A Duration: 5 to 7 Minutes

🎯 Top 10 Anticipated Faculty Examiner Questions & Architect's Model Answers

Question 1: "Under the CAP theorem, how can you claim that an offline system can continue operating if consistency is compromised?"

Examiner's Goal: Testing whether you understand the fundamental trade-off between Consistency (C) and Availability (A) during a network partition (P).

Architect's Defending Answer:

> *"Respected Examiner, we do not claim Strong Consistency (Linearizability) during a network partition. Under Eric Brewer's CAP theorem, network partitions are an unavoidable physical reality in distributed systems. Therefore, our system explicitly chooses* ***AP (Availability over Consistency)*** *during a partition.*
> *Because transactional retail operations—such as placing an order or printing a kitchen ticket—are mathematically* ***append-only events****, accepting them locally guarantees 100% operational*

> *availability and zero cashier latency. We trade off immediate global linearizability for **Eventual Consistency**, where all nodes converge to identical states once the partition heals via idempotent background synchronization."*

Question 2: "What happens if two cashiers on two different offline terminals take an order for the exact same physical Table 4 at the exact same second?"

Examiner's Goal: Probing how the system handles write-write conflicts on shared resources.

Architect's Defending Answer:

> *"This is an excellent operational question. We decouple the **Order Document** from the **Physical Table State**.*
> *1. Both orders are accepted with unique UUIDv4 idempotency keys and local ticket IDs. Neither cashier's terminal freezes, and both tickets are printed to the kitchen.*
> *2. Upon network reconciliation, the database commits both order records without data loss.*
> *3. For table occupancy state, the backend applies an **Append-Only Merging Rule**: both orders are associated with Table 4 as running KOT tickets. If the restaurant operates in strict pre-allocated table mode, the server flags the conflict in the manager's audit log for manual split or merge settlement. Crucially, **zero customer financial data or food orders are ever lost or dropped**."*

Question 3: "Why did you use IndexedDB instead of standard LocalStorage or WebSQL?"

Examiner's Goal: Testing your knowledge of browser storage engines and web standards.

Architect's Defending Answer:

> *"We selected IndexedDB because of three critical technical factors:*
> *1. **Synchronous Blocking vs. Asynchronous I/O**: LocalStorage is strictly synchronous. Writing a 50KB cart state blocks the browser's single V8 UI execution thread, causing visible UI frame drops and input lag. IndexedDB is completely asynchronous, executing in a background browser thread.*
> *2. **Storage Capacity**: LocalStorage is hard-capped at 5 megabytes across all origins. In contrast, Chromium browsers allocate up to 60% of available physical disk space (often tens of gigabytes) to IndexedDB.*
> *3. **Indexing & Search**: LocalStorage is a flat key-value string store. IndexedDB is a structured object database supporting B-tree compound indexing on fields like `[tenant_id, branchId, synced]`, allowing high-speed range queries.*

> *WebSQL was formally deprecated by the W3C and has been removed from modern browser engines."*

Question 4: "If the browser crashes or the cashier clears their browser cache, will all offline orders be permanently lost?"

Examiner's Goal: Checking data durability and browser lifecycle knowledge.

Architect's Defending Answer:

> *"No, sir. We implement two architectural safeguards:*
> *1. **Immediate Write-Ahead Persistence**: Every order is durably written to IndexedDB before the UI form resets. In modern OS engines, IndexedDB writes are committed directly to disk files (LevelDB in Chrome).*
> *2. **Persistent Storage API**: Standard browser temporary storage can theoretically be evicted by the OS during low disk states. To prevent this, our initialization script calls `navigator.storage.persist()`. When granted, the browser legally categorizes the application data as 'Persistent', barring the operating system or browser from auto-clearing it. Manual user clearing through Chrome settings can only be performed by an administrative user with device access, which is strictly restricted on commercial POS terminals via kiosk mode policies."*

Question 5: "How does your system prevent duplicate orders if a network request fails halfway through?"

Examiner's Goal: Testing understanding of the Two-Generals Problem and idempotency.

Architect's Defending Answer:

> *"We solve the Two-Generals Problem through **Client-Generated RFC4122 UUIDv4 Idempotency Keys**.*
> *At the exact millisecond the cashier clicks 'Submit', the client generates a unique UUIDv4 token and attaches it to the order payload before storing it in IndexedDB.*
> *If the server receives the order, writes it to PostgreSQL, but the network drops before the HTTP response reaches the client, the client's background worker retries the identical payload 5 seconds later. The FastAPI backend inspects the idempotency key in an atomic cache. Upon recognizing the key, the server rejects duplicate execution, returns the original cached response, and acknowledges the client, ensuring zero double billing."*

Question 6: "What is the difference between CRDTs and the Append-Only Log approach used in your architecture?"

Examiner's Goal: Testing deep distributed systems theory.

Architect's Defending Answer:

> *"Conflict-Free Replicated Data Types (CRDTs) are mathematically designed to commute—meaning updates can arrive in any order and automatically converge to identical state without a central coordinator (for example, collaborative text typing in Google Docs).*
> *While elegant, CRDTs introduce significant memory and computational overhead for complex relational business rules such as GST tax splits, inventory depletion, and credit balances. In enterprise financial software, an **Append-Only Operational Log** is superior because transactions are immutable business facts. Instead of attempting to mathematically merge conflicting edits to an existing invoice, we append new operational events (e.g., OrderPlaced, ItemAdded, TicketVoided), preserving an immutable audit trail that satisfies government financial compliance."*

Question 7: "How does the server handle sequential order tokens like `#001`, `#002` when multiple cashiers are offline?"

Examiner's Goal: Testing how atomic counters function when disconnected from a central database.

Architect's Defending Answer:

> *"When offline, terminals display a deterministic local sequence token with a terminal-specific prefix, such as `T1-042` or `ORD-OFFLINE-UUID`. This allows the kitchen staff to immediately identify the ticket on paper slips.*
> *Upon background reconciliation with PostgreSQL, the central database generates the official atomic daily sequential sequence (`#042`) using an exclusive row lock (`select(DailyOrderSequence).with_for_update()`). The local client is updated asynchronously with this official server token, ensuring zero sequence collisions across multiple concurrent registers."*

Question 8: "Can malicious users manipulate their local IndexedDB database using Chrome DevTools to alter prices?"

Examiner's Goal: Testing cybersecurity and client-side trust assumptions.

Architect's Defending Answer:

> *"The golden rule of distributed security is: **Never trust the client**.*
> *While a user can view or edit JavaScript memory in DevTools, the backend server does NOT accept prices or tax totals blindly from the client payload. The client sends menu item IDs and quantities; the server re-queries the authoritative PostgreSQL price master and recalculates taxes within the trusted backend environment. Furthermore, all requests must be signed with a valid JWT, and Row-Level Security restricts writes strictly to that authenticated tenant."*

Question 9: "How does your architecture behave when scaling to 10,000 simultaneous restaurants?"

Examiner's Goal: Testing scalability awareness.

Architect's Defending Answer:

> *"Because our architecture is local-first, it scales exponentially better than traditional cloud architectures:*
> *1. **80% Bandwidth & CPU Reduction**: Local terminals absorb 100% of the active UI rendering and typing load. The central server only receives lightweight JSON payloads when transactions finalize.*
> *2. **Connection Pooling with PgBouncer**: PostgreSQL database connections are pooled via PgBouncer in transaction mode.*
> *3. **Distributed Redis Backplane**: For multi-container horizontal scaling, WebSockets utilize Redis Pub/Sub channels (`branch:{id}:kds`) to broadcast tickets across multiple server nodes without cross-talk."*

Question 10: "What are the limitations of this offline-first architecture?"

Examiner's Goal: Testing academic objectivity and self-critique.

Architect's Defending Answer:

> *"The primary limitation is **Real-Time Global State Visibility**. For example, global inventory stock levels (e.g., 'Only 2 bottles of rare wine remaining') cannot be known with 100% certainty across two offline terminals simultaneously without network connectivity. In our design, we handle this by distinguishing between critical business operations (which must never stop) and advisory stock alerts (which update upon network reconnection)."*

Quick Tips for High Marks (Evaluation Rubric)

Speak with poise and precision: Never say "it just works"; explain the mechanism ("It uses non-blocking IndexedDB writes with UUIDv4 deduplication").

Refer directly to peer-reviewed papers: Cite Kleppmann (2019) and Brewer (2012).